

Heaven's Light is Our Guide
Rajshahi University of Engineering and Technology



Course Code

ECE 4124

Course Title

Digital Signal Processing Sessional

Experiment Date: September 13, 2025

Submission Date: October 11, 2025

Lab Report 4:

Performing DFT to compute Convolution Between Two Signals

Submitted to

Md. Faysal Ahamed
Assistant Professor
Dept of ECE, RUET
&
Moloy Kumar Ghosh
Lecturer
Dept of ECE, RUET

Submitted by

Md. Tajim An Noor
Roll: 2010025

Contents

Theory	1
Direct Convolution	1
Convulation Using DFT	1
Used Tools	2
Code & Output	2
Code	2
Output	3
Discussion	3
Conclusion	3
References	3

Performing DFT to Compute Convolution Between Two Signals

Theory

Direct Convolution

Convolution is a mathematical operation used to combine two signals to produce a third signal that expresses how the shape of one is modified by the other. In discrete-time signal processing, the convolution of two sequences $x[n]$ and $h[n]$ is defined as:

$$y[n] = (x * h)[n] = \sum_{k=0}^{N-1} x[k] \cdot h[n - k] \quad (1)$$

where N is the length of the sequences. This process involves sliding one signal over another, multiplying and summing the overlapping values [1].

Process:

1. Reverse one of the sequences (usually $h[n]$).
2. Shift it across the other sequence ($x[n]$).
3. For each shift, multiply overlapping values and sum them to get each output value.

Example: Let $x[n] = [1, 2, 3]$ and $h[n] = [4, 5]$. The convolution is:

$$\begin{aligned} y[0] &= 1 \cdot 4 = 4 \\ y[1] &= 1 \cdot 5 + 2 \cdot 4 = 5 + 8 = 13 \\ y[2] &= 2 \cdot 5 + 3 \cdot 4 = 10 + 12 = 22 \\ y[3] &= 3 \cdot 5 = 15 \end{aligned}$$

So, $y[n] = [4, 13, 22, 15]$.

Convolution Using DFT

Convolution can also be performed using the Discrete Fourier Transform (DFT) due to the Convolution Theorem, which states that convolution in the time domain corresponds to multiplication in the frequency domain [2].

Process:

1. Zero-pad both sequences to the same length (usually $N = N_x + N_h - 1$).
2. Compute the DFT of both sequences: $X[k] = \text{DFT}(x[n])$, $H[k] = \text{DFT}(h[n])$.
3. Multiply the DFTs pointwise: $Y[k] = X[k] \cdot H[k]$.
4. Compute the inverse DFT (IDFT) of $Y[k]$ to obtain the convolution result in the time domain.

Example: Given $x[n] = [1, 2, 3]$ and $h[n] = [4, 5]$, zero-pad both to length 4:

$$x[n] = [1, 2, 3, 0]$$

$$h[n] = [4, 5, 0, 0]$$

Compute DFTs, multiply, and take IDFT to get $y[n] = [4, 13, 22, 15]$ (same as direct convolution).

Summary: Direct convolution is straightforward for short signals, while DFT-based convolution is efficient for long signals due to fast algorithms like FFT [1, 2].

Used Tools

- Matlab Online
- LaTeX

Code & Output

Code

```
1  clc;
2  clear;
3  close all;
4
5  % Signals
6  x = [-2 4 2 1 5];
7  h = [5 -4 2 1 0];
8
9  % Length of convolution
10 N = length(x) + length(h) - 1;
11
12 % Zero-pad both signals to length N
13 x_pad = [x, zeros(1, N-length(x))];
14 h_pad = [h, zeros(1, N-length(h))];
15
16 % DFT of both signals
17 X = fft(x_pad);
18 H = fft(h_pad);
19
20 % Multiply in frequency domain
21 Y = X .* H;
22
23 % IDFT to get convolution result
24 y_dft = ifft(Y);
25
26 % Direct convolution (for verification)
27 y_direct = conv(x, h);
28
29 %% Display results
30 disp('Convolution using DFT:');
31 disp(y_dft);
32 disp('Direct convolution:');
33 disp(y_direct);
34
35 %% Visualization
36 n_x = 0:length(x)-1;
37 n_h = 0:length(h)-1;
38 n_y = 0:N-1;
39
40 figure;
41
42 subplot(4,1,1);
43 stem(n_x, x, 'r', 'filled', 'LineWidth',
44      ↪ 1.5);
45 title('Input Signal1 x(n)');
46 xlabel('n'); ylabel('Amplitude'); grid on;
47
48 subplot(4,1,2);
49 stem(n_h, h, 'b', 'filled', 'LineWidth',
50      ↪ 1.5);
51 title('Input Signal2 h(n)');
52 xlabel('n'); ylabel('Amplitude'); grid on;
53
54 subplot(4,1,3);
55 stem(n_y, real(y_dft), 'g', 'filled',
56      ↪ 'LineWidth', 1.5);
57 title('Convolution y(n) using DFT');
58 xlabel('n'); ylabel('Amplitude'); grid on;
59
60 subplot(4,1,4);
61 stem(n_y, y_direct, 'm', 'filled',
62      ↪ 'LineWidth', 1.5);
63 title('Convolution y(n) using Direct
64      ↪ conv()');
65 xlabel('n'); ylabel('Amplitude'); grid on;
```

Output

Convolution using DFT:

-10.0000 28.0000 -10.0000 3.0000 29.0000 -16.0000 11.0000 5.0000 -0.0000

Direct convolution:

-10 28 -10 3 29 -16 11 5 0

Figure 1: Console Output

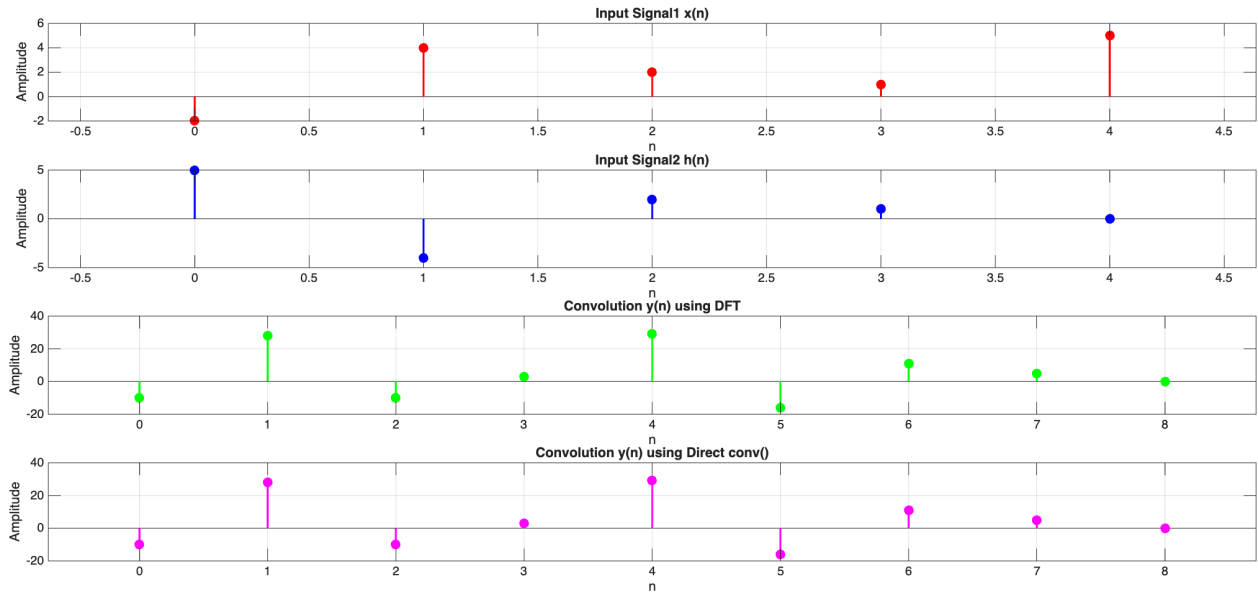


Figure 2: Plot Output

Discussion

In this experiment, the convolution of two discrete signals was computed using both the direct method and the DFT-based approach. The results obtained from both methods were observed to be identical, confirming the validity of the Convolution Theorem. The DFT-based method was facilitated by zero-padding the input sequences, transforming them into the frequency domain, performing pointwise multiplication, and then applying the inverse DFT. The efficiency of the DFT-based convolution was highlighted, especially for longer signals, where computational advantages were realized due to the use of FFT algorithms. The outputs and plots generated by Matlab were used to verify the correctness of the implementation.

Conclusion

The convolution between two signals was successfully performed using both direct computation and the DFT-based method. The equivalence of the results demonstrated the practical application of the Convolution Theorem in digital signal processing. The experiment illustrated that while direct convolution is suitable for short signals, the DFT-based approach is preferred for longer signals due to its computational efficiency. The use of Matlab and LaTeX facilitated the analysis and presentation of the results.

References

- [1] A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 2nd ed. Upper Saddle River, NJ: Prentice Hall, 1999.
- [2] J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, 4th ed. Upper Saddle River, NJ: Pearson Prentice Hall, 2007.